E-Commerce Website Manual Testing: Complete Guide & Interview Prep

A Step-by-Step Practical Handbook Written in Easy, Simple Language for QA Beginners

Website Tested: SauceDemo (saucedemo.com)	**Testing Type:** Manual, Functional, Negative	**Pass Rate:** 90.00% (27 Passed, 3 Failed)
User Tested: standard_user / secret_sauce	**Tools Used:** Browser, Google Sheets/Excel, GitHub	**Bugs Found:** 6 Real Defect Reports

1. What Is This Project About? (In Simple Words)

Imagine you are shopping on an online store like Amazon or Flipkart. Your goal is simple: log in, pick a product, add it to your cart, fill in your delivery address, check the total bill, and pay. As a **Manual QA Tester**, your job is NOT to build this website. Your job is to **test every single step** to check whether it works smoothly for real users, and to catch mistakes (bugs) before customers get frustrated.

The Shopping Journey We Tested:

1. **Login:** Entering username and password to enter the store.
2. **Product Catalog:** Viewing 6 items with their photos, descriptions, and prices.
3. **Sorting:** Changing product order (Name A-Z, Z-A, Price Low-High, Price High-Low).
4. **Shopping Cart:** Adding products, checking the red badge counter (1, 2, 3), and deleting items.
5. **Checkout Step 1:** Filling in First Name, Last Name, and Postal/Zip code.
6. **Checkout Step 2:** Verifying price calculations (Subtotal + 8% Sales Tax = Final Grand Total).
7. **Confirmation:** Reaching the 'Thank you for your order!' screen and returning home.

2. The 7 Project Stages Explained Step-by-Step

- **Stage 1: Exploring the Website First:** Never write tests blindly. First, open https://www.saucedemo.com. Click around like a real user. See what happens when you add items, when you leave fields blank, or when you type wrong passwords. Once you understand the normal flow, you are ready to write tests.
- **Stage 2: Creating the Test Plan:** A Test Plan is simply a roadmap. It answers: What are we testing? (Login, cart, checkout). What are we NOT testing? (Payment gateways, backend databases). What tools will we use? (Chrome, Excel). When are we done? (When all 30 tests are run and the pass rate is at least 85%).
- **Stage 3: Writing 30 Clear Test Cases:** A test case is like a cooking recipe with steps and an expected outcome. For example: Step 1 - Enter valid username. Step 2 - Enter valid password. Step 3 - Click Login. Expected Result: User lands on products page.
- **Stage 4: Executing Tests on the Live Website:** You open the website, follow the steps exactly, and write down what REALLY happened (Actual Result). If expected matches actual, it is a PASS. If not, it is a FAIL. Out of 30 tests, 27 Passed and 3 Failed (giving a 90% Pass Rate).
- **Stage 5: Reporting Genuine Defects (Bugs):** When a test fails, you write a bug report. You explain: What went wrong? How to reproduce it step-by-step? How serious is it? We found 6 real defects (such as being allowed to checkout with 0 items in the cart!).
- **Stage 6: Writing the Final Test Execution Report:** A summary for managers showing total tests run, how many passed, how many failed, what bugs were found, and our final recommendation: The website is mostly good, but 2 major bugs must be fixed before public launch.
- **Stage 7: Publishing Your Work on GitHub:** We uploaded the Test Plan, Test Cases, Bug Reports, Excel files, and PDF to GitHub so recruiters can click the link and see your proof of work immediately.

3. Important Testing Concepts Explained in Plain English

A. Equivalence Class Partitioning (ECP) — 'Group Similar Inputs':

Instead of testing 1,000 different wrong passwords, you group inputs into buckets. Bucket 1: Valid credentials (standard_user / secret_sauce). Bucket 2: Wrong password (standard_user / wrong123). Bucket 3: Empty inputs (leaving username or password blank). Testing just one from each bucket gives 100% confidence without wasting time.

B. Boundary Value Analysis (BVA) — 'Test at the Edges':
Errors usually happen at the boundaries (limits). For example, the boundary of a shopping cart is **0 items** vs. **1 item**. When we tested 0 items, we caught Bug BUG-02: the app let us click 'Checkout' even though the cart had nothing in it! Another boundary is price: verifying the lowest price ($7.99 Onesie) up to the highest price ($49.99 Fleece Jacket).

C. Negative Testing — 'Trying to Break the App':
Positive testing is entering correct data. Negative testing is entering wrong data on purpose: typing symbols in postal code (!@#$%), leaving mandatory fields empty, or trying to access /inventory.html without logging in. The app must gracefully show an error banner instead of crashing.

D. Financial Tax Calculation Math (8.00% Tax):
E-commerce apps must calculate taxes accurately to 2 decimal places. We checked SauceDemo's tax formula:
• We picked 2 items: Backpack ($29.99) + Bike Light ($9.99) = **$39.98 Subtotal**
• Tax at 8.00% = 39.98 * 0.08 = 3.1984 -> Rounded to 2 decimals = **$3.20 Tax**
• Grand Total = $39.98 + $3.20 = **$43.18**
The website showed exactly $39.98, $3.20, and $43.18. This proved the financial calculation was 100% accurate.

4. Quick Summary of the 30 Test Cases

Range	Module	What We Tested	Result
TC01 - TC08	Login & Auth	Valid login, wrong password, blank username, blank password, locked user, password masking, logout	8 / 8 Pass
TC09	Security	Direct URL access to /inventory.html without logging in (Redirects to login with error)	1 / 1 Pass
TC10 - TC16	Catalog & Sort	6 items display, 4 sorting options (A-Z, Z-A, Low-High, High-Low), item details page, search bar check	6 Pass, 1 Fail (No search bar)
TC17 - TC22	Shopping Cart	Add single/multi items, cart badge count (1->2->3), remove from catalog, remove from cart, page reload persistence	6 / 6 Pass
TC23 - TC27	Checkout Form	Empty cart checkout, missing First Name, missing Last Name, missing Zip, special characters in Zip	3 Pass, 2 Fail (Empty cart & Zip regex)
TC28 - TC30	Tax & Complete	Cancel checkout, 8% tax and subtotal math verification, finish order, thank you banner, Reset App State	3 / 3 Pass

5. The 6 Genuine Bugs We Found (Explained Simply)

- **BUG-01: No Search Bar Available (Medium Severity):** In any real shopping site, customers expect a search box at the top to type 'shirt' or 'bag'. In SauceDemo, there is no search bar at all. Customers can only sort with a dropdown.
- **BUG-02: User Can Checkout with an Empty Cart (High Severity):** If your cart has 0 items, the 'Checkout' button should be greyed out (disabled). In SauceDemo, you can click Checkout, enter your address, and place an order for $0.00! In real life, this creates ghost orders and wastes warehouse time.
- **BUG-03: Postal Code Accepts Weird Symbols like !@#$% (Medium Severity):** The Zip Code field accepts any random characters like '!@#$%^&*()'. It does not check if the postal code is actually numbers or valid letters. This can cause shipping delivery packages to get lost.
- **BUG-04: 'Remove' Button Doesn't Work on Inventory Page for problem_user (High Severity):** SauceDemo provides a test user named 'problem_user'. When you add an item and click 'Remove' on the catalog page, nothing happens! The button stays 'Remove' and the item remains in the cart.
- **BUG-05: Broken Product Photos Showing Dog Pictures (Low Severity):** For 'problem_user', all product photos fail to load and show the exact same photo of a dog with a camera. This looks unprofessional and confuses buyers.
- **BUG-06: Last Name Field is Frozen and Cannot Be Typed (Critical Severity):** For 'problem_user', you cannot type your Last Name during checkout. Because Last Name is required, the user is permanently blocked and can NEVER complete their purchase.

6. Top QA Interview Questions & Answers (Speak Like a Pro)

1. Tell me about this E-Commerce testing project.

How to Answer: I tested an online shopping website called SauceDemo. I tested the complete customer journey from login, to searching and sorting products, adding items to the cart, filling in the checkout form, verifying the 8% tax calculation, and finishing the order. I wrote 30 test cases, ran them all, achieved a 90% pass rate, and found 6 real bugs—including a major bug where users can checkout with 0 items in their cart.

2. What is Boundary Value Analysis? Did you use it?

How to Answer: Boundary Value Analysis means testing the edges or limits of an input, because developers most often make mistakes at boundaries. I used it on the shopping cart: the minimum allowed boundary to checkout should be 1 item. When I tested 0 items (below boundary), I discovered that the website still allowed me to checkout, which is a real bug!

3. What is the difference between Severity and Priority? Give examples from your project.

How to Answer: Severity is how badly the bug damages the system technically. Priority is how quickly developers need to fix it.
Example of High Severity + High Priority: When using problem_user, the Last Name field freezes, blocking checkout completely. No customer can buy anything (Critical/P1).
Example of Medium Severity + Medium Priority: The missing search bar (BUG-01). Users can still buy items using sorting, so it's not blocking sales immediately, but it hurts usability.

4. How did you verify the price and tax calculation?

How to Answer: I picked two products: Sauce Labs Backpack ($29.99) and Bike Light ($9.99). Their subtotal was $39.98. The website charges 8% tax. I calculated 39.98 * 0.08 = 3.1984. Rounding to 2 decimal places gives $3.20 tax. Adding $39.98 + $3.20 gave a grand total of $43.18. I compared this with the screen, and the website matched my math with 100% precision.

5. What will you do if a developer says 'The empty cart checkout is not a bug'?

How to Answer: I will politely explain the business impact: allowing orders with 0 items creates $0 invoices, triggers automatic confirmation emails to customers, and creates confusing orders in the warehouse logistics queue. If we still disagree, I will bring the issue to the Product Owner with clear steps and let them decide based on business rules.

6. What is the difference between Smoke Testing and Regression Testing?

How to Answer: Smoke Testing is a quick check to see if basic core features work before doing deep testing (e.g., Can the user log in and view products?). Regression Testing is re-running old tests after a bug fix to make sure the fix didn't break anything that used to work.

7. What are the essential fields in a good Bug Report?

How to Answer: A great bug report needs: 1. Bug ID, 2. Clear Title/Summary, 3. Severity and Priority, 4. Environment (Browser/OS), 5. Step-by-step reproduction instructions, 6. Expected Result, 7. Actual Result, and 8. Screenshots or video proof.

7. Resume Bullet Points & 30-Second Elevator Pitch

4 Concise Resume Bullet Points (Copy & Paste):

- **Executed 30 manual test cases** for an e-commerce platform covering login, catalog sorting, cart management, and checkout, achieving a **90% pass rate**.
- **Applied ECP and BVA techniques** to validate form inputs, negative edge cases, and verify **8.00% sales tax and price calculations**.
- **Identified and logged 6 defects in Jira format** with clear reproduction steps, including a high-severity bug allowing checkout with an empty cart.
- **Authored full QA documentation**—including an IEEE 829 Test Plan, test matrices, and execution reports—published on GitHub.

30-Second Elevator Pitch (For HR / Recruiter Phone Calls):

"I recently completed an end-to-end manual QA testing project on an e-commerce website. I designed and executed 30 test cases covering authentication, product sorting, cart operations, form validation, and tax calculations. I applied testing methods like Equivalence Partitioning and Boundary Value Analysis to find edge cases, and I documented 6 real bugs in Jira format. All my test plans, Excel sheets, and reports are published on my GitHub. I am ready to bring this structured testing approach to your QA team!"